Cluster and Cloud Computing Assignment 1 – Big Data Analytics

Problem Description

Your task in this programming assignment is to implement a parallelized application leveraging the University of Melbourne HPC facility SPARTAN. Your application will process two large social media datasets from Mastodon and BlueSky. The objective is to identify the languages that people use for posting.

You should be able to log in to SPARTAN through running the following command:

ssh your-unimelb-username@spartan.hpc.unimelb.edu.au

with the password you set for yourself on *karaage* (https://dashboard.hpc.unimelb.edu.au/karaage). Thus, I would log in as:

ssh rsinnott@spartan.hpc.unimelb.edu.au
password = my karaage password (not my UniMelb password)

If you are a Windows user then you may need to install an application like Putty.exe to run *ssh*. (If you are coming from elsewhere with different firewall rules, then you may need to use a VPN).

The files to be used in this assignment are accessible at:

- /data/gpfs/projects/COMP90024/mastodon-large.ndjson
- /data/gpfs/projects/COMP90024/bluesky-large.ndjson
  - These are the main large files to use for your final analysis and report write up, i.e., **do not use these files for software development and testing**.
- /data/gpfs/projects/COMP90024/mastodon-medium.ndjson
- /data/gpfs/projects/COMP90024/bluesky-medium.ndjson
  - are much smaller JSON files that should be used for testing.
- /data/gpfs/projects/COMP90024/mastodon-small.ndjson
- /data/gpfs/projects/COMP90024/bluesky-small.ndjson
  - are the smallest JSON files that should be used for initial testing and exploring the data.

You may decide to use the smaller JSON files on your own PC/laptop to start with for development and testing. The small/medium files will be made available through Canvas. As noted in the lecture/workshop, we cannot help you with the setup of software on your own local (PC/laptop).

Once on SPARTAN, you should make a symbolic link to these files, i.e., you should run the following commands at the Unix prompt **from your own user directory on SPARTAN**:

```
ln –s /data/gpfs/projects/COMP90024/mastodon-large.ndjson
ln –s /data/gpfs/projects/COMP90024/bluesky-large.ndjson
ln –s /data/gpfs/projects/COMP90024/mastodon-medium.ndjson
ln –s /data/gpfs/projects/COMP90024/bluesky-medium.ndjson
ln –s /data/gpfs/projects/COMP90024/mastodon-small.ndjson
ln –s /data/gpfs/projects/COMP90024/bluesky-small.ndjson
```

(Do not cut and paste these commands as it will include the carriage return. Type them in manually).

Once done you should see something like the following **in your home directory**:

```
lrwxrwxrwx 1 rsinnott punim1502 50 Mar 7 11:43 mastodon-large.ndjson -> /data/gpfs/projects/COMP90024/mastodon-large.ndjson
lrwxrwxrwx 1 rsinnott punim1502 50 Mar 7 11:43 bluesky-large.ndjson -> /data/gpfs/projects/COMP90024/bluesky-large.ndjson
lrwxrwxrwx 1 rsinnott punim1502 49 Mar 7 11:43 mastodon-medium.ndjson -> /data/gpfs/projects/COMP90024/mastodon-medium.ndjson
lrwxrwxrwx 1 rsinnott punim1502 49 Mar 7 11:43 bluesky-medium.ndjson -> /data/gpfs/projects/COMP90024/bluesky-medium.ndjson
lrwxrwxrwx 1 rsinnott punim1502 51 Mar 7 11:43 mastodon-small.ndjson -> /data/gpfs/projects/COMP90024/mastodon-small.ndjson
lrwxrwxrwx 1 rsinnott punim1502 51 Mar 7 11:43 bluesky-small.ndjson -> /data/gpfs/projects/COMP90024/bluesky-small.ndjson
```

Your assignment is to (*eventually*!) search the large Mastodon and BlueSky data files and identify the languages (*langs /language* attribute) used for posting social media content and the number of posts coded with that language attribute. If a post has multiple languages, e.g ["*En*""*Es*"] then these should both be counted to the language counts, i.e. +1 English and +1 Spanish. The results should look something like the table below (noting that the numbers are representative/dummy):

Mastodon Languages Used	**Frequency of Occurrence (#posts)**	**BlueSky Languages Used**	**Frequency of Occurrence (#posts)**
En (English)	20,000,000	*En (English)*	30,000,000
De (German)	1,000,000	*Zh (Chinese)*	1,000,000
Fr (French)	800,000	*Yi (Yiddish)*	300,000
Jp (Japanese)	500,000	*Uk (Ukrainian)*	200,000
It (Italian)	400,000	*Tk (Turkish)*	150,000
Pt (Portuguese)	300,000	*Pt (Portuguese)*	140,000
Zh (Chinese)	200,000	*Sv (Swedish)*	120,000
Es (Spanish)	100,000	*Es (Spanish)*	90,000
Th (Thai)	50,000	*Ru (Russian)*	75,000

The language codes are based on ISO-639 (see https://en.wikipedia.org/wiki/List_of_ISO_639_language_codes).

Your code should deal with potentially ill-formatted JSON posts and potentially other issues, e.g. posts with no language attribute (e.g. *Null*) or language attributes not in the ISO-639 list.

You should implement a single program for processing both the Mastodon and BlueSky data, i.e. not a Python script for processing Mastodon and a separate script for processing BlueSky.

Your application should allow a given number of nodes and cores to be utilized. Specifically, **your application should be run once** to search the *largest NDJSON* files on each of the following resources:

- 1 node and 1 core;
- 1 node and 8 cores;
- 2 nodes and 8 cores (with 4 cores per node).

The resources should be set when submitting the search application with the appropriate *SLURM* options. Note that you should run a single *SLURM* job three separate times on each of the resources given here, i.e. you should not need to run the same job 3 times on 1 node 1 core for example to benchmark the application. (This is a shared facility and this many COMP90024 students will consume a lot of resources especially with the large file!).

You can implement your solution using any routines and libraries you wish however it is strongly recommended that you follow the guidelines provided on access and use of the SPARTAN cluster. Do not for example think that the job scheduler/SPARTAN automatically parallelizes your code – it doesn't! You may wish to use the pre-existing MPI libraries that have been installed for C, C++ or Python, e.g., mpi4py. You should feel free to make use of the Internet to identify which JSON processing libraries you might use. You may also use any regular expression libraries that you might need for string comparison.

Your application should return the final results and the time to run the job itself, i.e. the time for the first job starting on a given SPARTAN node to the time the last job completes. You may ignore the queuing time. The focus of this assignment is not to optimize the application to run faster, but to learn about HPC and how basic benchmarking of applications on a HPC facility can be achieved and the lessons learned in doing this on a shared resource.

A Word About Large Language Models

Students are allowed to use LLMs in this assignment. However, any/all use of an LLM should be declared and documented, e.g. the LLM selected, the prompts that were used and a description of the development process in using the LLM. You may also discuss the different solutions generated by the LLM and why you selected the one that you ultimately decided upon. Any LLM solutions should all be generated/tested on the small and medium files only. It is noted that the final exam may well focus on details of the implementation that require insight into for example the use of MPI. Thus, developing the application yourself is beneficial compared to just using the LLM for all code generation (there will be no LLM access in the exam as it is a closed book exam written on paper!)

Final packaging and delivery

You should write a brief report on the application – **no more than 4 pages**!, outlining how it can be invoked, i.e. it should include the scripts used for submitting the job to SPARTAN, the approach you took to parallelize your code,

and describe variations in its performance on different numbers of nodes and cores. Your report should include the actual results tables as outlined above and a single graph (e.g., a bar chart) showing the time for execution of your solution on 1 node with 1 core, on 1 node with 8 cores and on 2 nodes with 8 cores. You should relate your results back to Amdahl's law to describe the potential performance changes. If you used an LLM then an extra page should be added on how you used the LLM as described above, i.e. if you use an LLM then the report can be up to 5-pages.

Deadline

The assignment should be submitted to Canvas as a zip file. The zip file must be named with the students named in each team and their student Ids. That is, *ForenameSurname-StudentId:ForenameSurname-StudentId* might be *<SteveJobs-12345:BillGates-23456>.zip*. Only one report is required per student pair and only one student needs to upload this report. The deadline for submitting the assignment is: **Friday 17th April (by 12 noon!)**.

It is strongly recommended that you do not do this assignment at the last minute, as it may be the case that the Spartan HPC facility is under heavy load when you need it and hence it may not be available! You have been warned…!!!!

Marking

The marking process will be structured by evaluating whether the assignment (application + report) is compliant with the specification given. This implies the following:

- A working demonstration – **60% marks**
- Report and write up discussion – **40% marks**

Timeliness in submitting the assignment in the proper format is important. **A 10% deduction per day will be made for late submissions.**

You are free to develop your system where you are more comfortable with (at home, on your PC/laptop, in the labs, on SPARTAN itself - but not on the largest file until you are ready!). Your code should of course work on SPARTAN.